МИНИСТЕРСТВО ОБРАЗОВАНИЯ КУЗБАССА

ГПОУ «ЮРГИНСКИЙ ТЕХНОЛОГИЧЕСКИЙ КОЛЛЕДЖ»
ИМ. ПАВЛЮЧКОВА Г.А.
Отделение АИТ

СОЗДАНИЕ МОДУЛЬНЫХ ТЕСТОВ С ВНЕШНИМИ ДАННЫМИ

МЕТОДИЧЕСКИЕ УКАЗАНИЯ
ПО ПРАКТИЧЕКОЙ РАБОТЕ №2

МДК.01.03 ТЕСТИРОВАНИЕ ИНФОРМАЦИОННЫХ РЕСУРСОВ

Специальность 09.02.09 Веб-разработка

Составитель
Соболева Д. А.

2025 г.

Цель: научиться создавать модульные тесты с использованием внешних данных, освоить загрузку и преобразование данных из файлов CSV, JSON и других источников, закрепить навыки проверки функций и методов на различных входных значениях, освоить обработку исключений и подготовить основу для автоматизированного тестирования с масштабируемыми сценариями проверки корректности работы программного кода

Краткая теория

Модульное тестирование с внешними данными позволяет проверять функции и методы программы на большом количестве различных входных значений, которые хранятся вне кода теста. Такие данные могут поступать из файлов (CSV, JSON, Excel), баз данных или конфигурационных файлов.

Использование внешних данных делает тесты более масштабируемыми, упрощает их сопровождение и позволяет легко добавлять новые сценарии проверки без изменения кода теста. В Python для этого часто применяют фреймворк **PyTest**, который поддерживает параметризацию тестов через декоратор @pytest.mark.parametrize и загрузку данных из внешних источников.

При создании тестов с внешними данными важно:

- корректно загружать и преобразовывать данные в нужный формат;
- обеспечивать независимость каждого тестового случая;
- проверять корректность обработки как обычных, так и граничных значений;
- учитывать возможность возникновения исключений и ошибок.

.

Задания:

1. Исправьте ошибку в тесте, использующем CSV-файл, продемонстрированном на рисунке 1.

```python
import csv
import pytest

def add(a, b):
    return a + b

def load_data():
    with open("data.csv") as f:
        reader = csv.reader(f)
        return [(row[0], row[1], row[2]) for row in reader]

@pytest.mark.parametrize("a,b,expected", load_data())
def test_add(a, b, expected):
    assert add(a, b) == expected
```

Рисунок 1 – Задание 1

2. Исправьте синтаксическую ошибку и приведите типы данных в тесте с JSON, продемонстрированном на рисунке 2.

```python
import json
import pytest

def multiply(a, b):
    return a * b

with open("data.json") as f:
    test_data = json.load(f)

@pytest.mark.parametrize("a,b,expected", test_data)  # ошибка в названии декоратора
def test_multiply(a, b, expected):
    assert multiply(a, b) == expected
```

Рисунок 2 - Задание 2

3. Найдите логическую ошибку в тесте, использующем данные из словаря, продемонстрированном на рисунке 3.

```python
import pytest

def divide(a, b):
    return a / b

test_cases = [
    {"a": 10, "b": 2, "expected": 5},
    {"a": 8, "b": 0, "expected": "Error"}
]

@pytest.mark.parametrize("case", test_cases)
def test_divide(case):
    assert divide(case["a"], case["b"]) == case["expected"]
```

Рисунок 3 - Задание 3

4. Исправьте тест, в котором внешние данные передаются через список кортежей, но неправильно указан тип, продемонстрированном на рисунке 4.

```python
import pytest

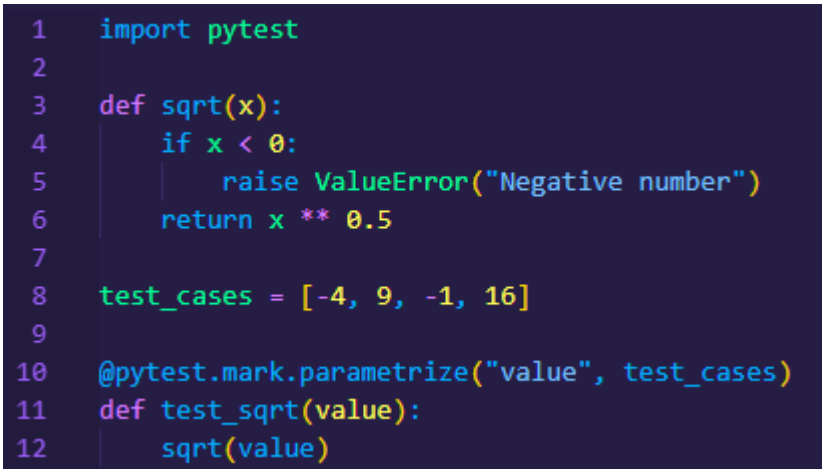
def subtract(a, b):
    return a - b

test_data = [
    ("10", "5", 5),
    ("20", "3", 17)
]

@pytest.mark.parametrize("a,b,expected", test_data)
def test_subtract(a, b, expected):
    assert subtract(a, b) == expected
```

Рисунок 4 - Задание 4

5. Исправьте тест с обработкой исключений, данные поступают из внешнего файла, продемонстрированном на рисунке 5.



```python
import pytest

def sqrt(x):
    if x < 0:
        raise ValueError("Negative number")
    return x ** 0.5

test_cases = [-4, 9, -1, 16]

@pytest.mark.parametrize("value", test_cases)
def test_sqrt(value):
    sqrt(value)
```

Рисунок 5 - Задание 5